

Computer Organization & Architecture

Course Code: 2501CS71

IE-2 Important Questions — Detailed Answers

Units III, IV & V

UNIT - III: CPU and Microprogrammed Control Unit

Q1a: Explain General Register Organization in a basic computer with block diagram.

Definition

General Register Organization refers to a CPU design where multiple general-purpose registers are available to perform arithmetic, logic and data transfer operations, unlike single-accumulator-based CPUs.

Key Points

- **General Register Set:** A set of n registers ($R_0, R_1, \dots, R_{n-1}$) is connected to an ALU via a common bus.
- **MUX / Bus:** A multiplexer (MUX) or bus structure selects the source and destination registers for each operation.
- **Control Unit Signals:** Control signals (SELA, SELB, SELD, OPR) are issued by the control unit to direct operations.
- **Result Storage:** ALU result can be stored back into any general-purpose register.

Block Diagram Description

The organization consists of:

1. MUX A — Selects source operand A from any register R_i
2. MUX B — Selects source operand B from any register R_i
3. ALU — Performs arithmetic/logic operation specified by OPR signal
4. MUX (output) — Routes ALU result to destination register via SELD
5. Register File — Array of registers R_0 to R_{n-1}

Control Fields in Instruction

Field	Bits	Purpose	Example
SELA	3	Select source A	R2
SELB	3	Select source B	R3
SELD	3	Select destination	R1
OPR	5	ALU operation	ADD

Advantages

- Reduces memory accesses — operands stay in registers
- Faster execution due to register-level operations
- Flexible — any register can be source or destination
- Supports pipelining efficiently

Q1b: Illustrate PUSH and POP operations in Stack Organization.

Stack Organization - Definition

A stack is a Last-In-First-Out (LIFO) storage structure. Operations are performed at one end called the TOP of the stack using PUSH (insert) and POP (delete) operations.

Registers Involved

- SP (Stack Pointer) — Points to the current top of stack
- FULL flag — Set to 1 when stack is full
- EMTY flag — Set to 1 when stack is empty
- DR (Data Register) — Holds data to be pushed/popped

PUSH Operation (Insertion)

Initially: SP = 0, FULL = 0, EMTY = 1

- $SP \leftarrow SP + 1$ (Increment stack pointer)
- $M[SP] \leftarrow DR$ (Store data at top of stack)
- If SP = 0 then FULL $\leftarrow$ 1, EMTY $\leftarrow$ 0 (stack is full - circular wrap)

POP Operation (Deletion)

Initially: FULL = 1, EMTY = 0, SP at top

- $DR \leftarrow M[SP]$ (Read data from top of stack)
- $SP \leftarrow SP - 1$ (Decrement stack pointer)
- If SP = 0 then FULL $\leftarrow$ 0, EMTY $\leftarrow$ 1

Types of Stack Organization

6. Register Stack — Uses registers as stack memory; faster but limited size
7. Memory Stack — Uses main memory; larger but slower

Stack in Memory Organization (Memory Stack)

- Stack region is separate from program and data regions
- Program: 1000–2000; Operands: 2000–3000; Stack: 3000–4001
- PC holds program address, AR holds operand address, SP holds stack address

Q2a: Differentiate Three, Two, One and Zero addressing formats with suitable examples.

Instruction Formats Overview

The number of address fields in an instruction determines its addressing format. Each format has different impacts on code size, memory access, and speed.

Format	Address Fields	Operation	Example	Remark
Three-Address	3 (dest, src1, src2)	dest = src1 op src2	ADD R1, R2, R3	Most flexible; long instructions
Two-Address	2 (dest/src1, src2)	dest = dest op src	ADD R1, R2	Overwrites one operand
One-Address	1 (operand only)	ACC = ACC op operand	ADD X	Uses Accumulator implicitly
Zero-Address	0 (stack-based)	Stack-based ops	ADD (pops, pushes)	Shortest code; uses stack

Three-Address Example: $A = (B + C) \times (D + E)$

- ADD R1, B, C $\rightarrow R1 = B + C$
- ADD R2, D, E $\rightarrow R2 = D + E$
- MUL A, R1, R2 $\rightarrow A = R1 \times R2$

Two-Address Example: Same Expression

- MOV R1, B
- ADD R1, C → $R1 = B + C$
- MOV R2, D
- ADD R2, E → $R2 = D + E$
- MUL R1, R2 → $R1 = A$

One-Address Example: Using Accumulator

- LOAD B
- ADD C → $ACC = B + C$
- STORE T
- LOAD D
- ADD E → $ACC = D + E$
- MUL T → $ACC = A$

Zero-Address Example: Stack-based

- PUSH B, PUSH C, ADD (→ B+C on top)
- PUSH D, PUSH E, ADD (→ D+E on top)
- MUL → pops both, pushes result
- POP A → stores final result

Q2b: Compare RISC and CISC Architectures.

RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Simple, fixed-size instructions	Complex, variable-size instructions
Executes in 1 clock cycle	May take multiple clock cycles
Few addressing modes	Many addressing modes
Hardware-based (Hardwired) control unit	Software-based (Microprogrammed) CU
Pipelining is easily implemented	Pipelining is complex
Large number of general registers	Few registers; memory used for storage
Examples: ARM, SPARC, MIPS, 8085, 8086	Examples: Intel x86, VAX, 8051
Faster for simple operations	Better for complex operations
ROM not used for control	ROM used for microprogram storage

Q3a: Classify instructions based on functionality — Data Transfer and Arithmetic Instructions.

Classification of Instructions

Instructions are classified into 3 types based on functionality:

8. Data Transfer Instructions
9. Data Manipulation Instructions (Arithmetic, Logical, Shift)
10. Program Control Instructions

1. Data Transfer Instructions

These instructions move data from one location to another without changing its content. Location may be Register-Register (R-R), Register-Memory (R-M), or Register-I/O.

Name	Mnemonic	Operation
Load	LD	$R \leftarrow M[EA]$
Store	ST	$M[EA] \leftarrow R$
Move	MOV	$R1 \leftarrow R2$
Exchange	XCH	Swap $R1 \leftrightarrow R2$
Input	IN	$R \leftarrow I/O \text{ Port}$
Output	OUT	$I/O \text{ Port} \leftarrow R$
Push	PUSH	Stack $\leftarrow R$
Pop	POP	$R \leftarrow \text{Stack}$

2. Arithmetic Instructions

Perform mathematical operations on data in registers or memory.

Name	Opcode	Description
Addition	ADD	$A + B \rightarrow \text{dest}$
Subtraction	SUB	$A - B \rightarrow \text{dest}$
Add with Carry	ADDC	$A + B + \text{Carry}$
Subtract with Borrow	SUBB	$A - B - \text{Borrow}$
Multiply	MUL	$A \times B$
Negate (2's complement)	NEG	$-A \rightarrow \text{dest}$
Increment	INC	$A = A + 1$
Decrement	DEC	$A = A - 1$

Q3b: Classify instructions — Logical and Shift Instructions.

Logical Instructions

Logical instructions perform bit-level operations on strings of bits or individual bits.

Name	Mnemonic	Operation
Clear	CLR	Set all bits to 0
Complement	COM	Invert all bits (1's comp)
AND	AND	Bitwise AND
OR	OR	Bitwise OR
Exclusive-OR	XOR	Bitwise XOR
Clear Carry	CLRC	Carry flag = 0
Set Carry	SETC	Carry flag = 1
Enable Interrupt	EI	Enable interrupts

Disable Interrupt	DI	Disable interrupts
-------------------	----	--------------------

Shift Instructions

Shift operations move bits left or right within a register.

Name	Mnemonic	Description
Logical Shift Right	SHR	0 inserted at MSB
Logical Shift Left	SHL	0 inserted at LSB
Arithmetic Shift Right	ASHR	Sign bit preserved at MSB
Rotate Right	ROR	LSB wraps to MSB
Rotate Left	ROL	MSB wraps to LSB
Rotate Right through Carry	RORC	LSB → Carry → MSB
Rotate Left through Carry	ROLC	MSB → Carry → LSB

Example Program Using Shift/Logic

- MOV R1, 2 → Load value 2 into R1
- INC R1 → R1 = 3
- COM R1 → R1 = complement of 3
- SHR R1 → Logical shift right by 1 bit

Q4a: Illustrate any five Addressing Modes with suitable examples.

Definition of Addressing Mode

An addressing mode defines the way in which the operand is specified in an instruction. Each instruction has an opcode, address field, and addressing mode bits.

1. Immediate Addressing Mode

- The operand value is directly specified in the instruction itself.
- No memory access needed to get the operand.
- Example: MOV A, 15h → A ← 15h (15h is the immediate operand)

2. Register Addressing Mode

- The operand is present in one of the CPU registers.
- Fastest mode (no memory access needed).
- Example: MOV R1, R2 → R1 ← R2 (value of R2 moved to R1)

3. Register Indirect Addressing Mode

- Register holds the memory address of the operand (not the operand itself).
- Denoted using square brackets [].
- Example: MOV R1, [5000H] → R1 ← M[5000H] (load data at address 5000H)

4. Direct Addressing Mode

- The operand is present at the address given in the instruction.
- Also called Absolute Addressing Mode (I = 0).
- Example: LDA [4000] → ACC ← M[4000]

5. Indexed Addressing Mode

- Effective Address (EA) = Base Address + Index Register value.
- Useful for accessing array elements.
- Formula: $EA = \text{Base Address} + \text{Index Register}$
- Example: If Base = 5000H, Index = 2000H $\rightarrow$ EA = 7000H

Summary Table

Mode	EA Calculation	Speed	Example
Immediate	Operand in instruction	Fastest	MOV A, 5
Register	Operand in register	Very Fast	MOV R1, R2
Register Indirect	$EA = [R]$	Fast	MOV R1, [R2]
Direct	EA = address in instruction	Moderate	LDA [4000]
Indexed	$EA = \text{Base} + \text{Index Reg}$	Moderate	EA = 5000+2000

Q4b: Explain Microprogrammed Control Organization with block diagram.

Definition

Microprogrammed Control Unit is a control unit where control signals are generated by executing a microprogram stored in a special ROM called Control Memory. It follows the CISC approach.

Key Components

- **Control Memory:** Control Memory (ROM) — Stores microoperations; made of ROM (non-volatile); only read operation allowed.
- **CAR:** Control Address Register (CAR) — Acts as the Program Counter (PC) for microinstructions; holds address of next microinstruction.
- **NAG:** Next Address Generator (NAG) — Responsible for identifying the address of next microinstruction.
- **CDR:** Control Data Register (CDR) — Holds the microinstruction fetched from control memory.
- **External Input:** External Input — Provides condition signals from other parts of the system.

Block Diagram Components

11. External Input $\rightarrow$ Next Address Generator (NAG)
12. NAG $\rightarrow$ Control Address Register (CAR)
13. CAR $\rightarrow$ Control Memory (ROM)
14. Control Memory $\rightarrow$ Control Data Register (CDR)
15. CDR $\rightarrow$ Control Unit Output + Next Address Info back to NAG

Working Principle

- Control memory stores the operands and opcodes to perform specific operations.
- The control unit fetches microinstruction from control memory using CAR address.
- Each microinstruction specifies one or more micro-operations.
- After execution, NAG computes the address of the next microinstruction.
- If SBR = 1 $\rightarrow$ subroutine function is called; if SBR = 0 $\rightarrow$ no call/return

Advantages

- Easy to modify — just update the microprogram in ROM
- Lower cost than hardwired CU
- Suitable for complex (CISC) instruction sets

Disadvantages

- Slower than hardwired control unit
- Pipelining is not present in microprogrammed CU

Q5a: Discuss Address Sequencing and how next microinstruction address is selected.

Definition

Address Sequencing is the process of identifying and fetching the address of the next microinstruction to be executed. The Next Address Generator (NAG) is responsible for this.

Four Ways for Next Address Identification

16. **1. Incrementing the CAR (Control Address Register)**
 - CAR is incremented by 1 to get the next sequential microinstruction address.
 - Used when instructions execute sequentially without any branching.
 - Example: MOV R1, 5 → MOV R2, 5 → ADD R1, R2 → END (sequential execution)
17. **2. Checking Subroutine Register (Call & Return)**
 - The next address depends upon call and return functions.
 - If SBR = 1 → subroutine is called; SBR = 0 → no subroutine
 - Example: MOV R1,2 → MOV R2,4 → CALL → Return → MUL R1,R2 → END
18. **3. Checking Conditional/Unconditional Branch**
 - Next address depends on a condition given in the program.
 - Branch logic checks the select status bit and branch address.
 - Example: If CMP R = 0 → Branch to 'even' else Branch to 'odd'
19. **4. Mapping Computer Instruction to Microprogrammed Instruction**
 - Used when a machine-level instruction is mapped to a sequence of microinstructions.
 - Mapping logic converts instruction code to microprogram starting address.

Block Diagram Elements

- Instruction Code → Mapping Logic
- Mapping Logic → Multiplexer
- Branch Logic → Multiplexer
- Subroutine Register → Multiplexer
- Multiplexer → CAR → Incrementer → Control Memory → Micro-operations

Q5b: Explain the fields of Microinstruction Format.

Definition

A microinstruction is the basic unit stored in Control Memory (ROM). Each microinstruction specifies one or more micro-operations such as arithmetic, logical, and shift operations.

Microprogrammed Computer Configuration (from Notes)

- Main Memory (MM): 2048×16 → 11 address lines, 16 data lines
- Control Memory (CM): 128×20 → 7-bit address (CAR has 7 bits), 20-bit data bus
- CAR (Control Address Register) acts as PC for microinstructions — holds 7-bit address of next microinstruction
- SBR (Subroutine Register): if SBR = 1 → subroutine functions are present; if SBR = 0 → no call and return functions
- Operation is performed using AR, DR, PC, AC registers

- CAR and SBR are responsible for performing microoperations

Fields of a Microinstruction (20-bit format)

20. F1, F2, F3 — Micro-operation fields
 - Encode arithmetic, logical, and shift micro-operations
 - Each field selects a specific micro-operation to perform on registers
 - Arithmetic: ADD, SUB, MUL | Logical: AND, OR, XOR | Shift: SHR, SHL
21. CD (Condition) — Condition field
 - Selects the status bit used for conditional branching
 - Checks conditions like carry, zero, overflow before branching
22. BR (Branch) — Branch field
 - Determines branching behavior of the microinstruction
 - Controls whether to jump, continue, call subroutine, or return
23. AD (Address) — Address field (7 bits)
 - Holds the branch address of next microinstruction (7-bit since CM has $128 = 2^7$ locations)
 - Used by the Next Address Generator (NAG) to compute next CAR value

Microinstruction Format Table (20-bit)

F1 (3 bits)	F2 (3 bits)	F3 (3 bits)	CD (2 bits)	BR (2 bits)	AD (7 bits)
ALU / Arith op	Data path op	Shift op	Condition select	Branch type	Next address

Note: Total bits: $3+3+3+2+2+7 = 20$ bits — matches Control Memory data bus width of 20 bits

Micro-operation Encoding (from Hardwired CU example in notes)

Decoder O/P	Opcode	Operand
D0	000	ADD
D1	001	SUB
D2	010	MUL
D3	011	DIV
D4	100	SHR
D5	101	AND
D6	110	CLR
D7	111	SHL

Example Microprogrammed Sequence (from Notes)

- MOV R1, 5 → Move value 5 into R1
 - MOV R2, 5 → Move value 5 into R2
 - ADD R1, R2 → Add R1 and R2, result in R1
 - END → Next instruction executed without any loops (sequential CAR increment)
 - Processor language is ALP (Assembly Language Programming)
-

UNIT - IV: Memory Organization and I/O Organization

Q1a: Discuss the Memory Hierarchy in a Computer System.

Definition

Memory Hierarchy is the arrangement of different types of memory in a computer system based on speed, cost, and storage capacity. Higher levels are faster but costlier and smaller.

Memory Hierarchy Levels (Top to Bottom)

24. CPU Registers — Fastest, highest cost, lowest storage (16–64 bits)
25. Cache Memory (L1, L2, L3) — Very fast, high cost, low storage; made of SRAM
26. Main Memory (RAM) — Moderate speed, moderate cost; made of DRAM
27. Magnetic Disk (HDD) — Slow, low cost, large storage
28. Optical Disk / Magnetic Tape — Slowest, lowest cost, largest storage

Key Relationships

- $\text{Speed} \propto \text{Cost}$ (higher speed = higher cost)
- $\text{Speed} \propto 1/\text{Storage}$ (higher speed = lower storage capacity)
- $\text{Cost} \propto 1/\text{Storage}$ (higher storage = lower cost per bit)

Characteristics of Memory Hierarchy

Characteristic	Top Level	Bottom Level	Trend
Capacity	Low	Very High	Increases downward
Access Time	Low (fast)	High (slow)	Increases downward
Cost per bit	High	Low	Decreases downward
Performance	High	Low	Decreases downward

Types of Memory

- **Main Memory:** Main Memory (RAM) — Stores user programs and OS; made of DRAM; primary memory on board
- **Auxiliary Memory:** Auxiliary Memory — Secondary storage (HDD, SSD, tape); non-volatile; stores large data
- **Cache Memory:** Cache Memory — Stores frequently used instructions; made of SRAM; types: L1, L2, L3
- **Associative Memory:** Associative Memory (CAM) — Content-based access; high performance; contains Argument (A), Key (K), Match (M) registers
- **Virtual Memory:** Virtual Memory — Illusion memory; used when program size > main memory size

Q1b: Explain the hardware organization of Associative Memory with block diagram.

Definition

Associative Memory (also called CAM — Content Addressable Memory) is a type of memory where data is accessed by its content rather than by its address. It offers high access performance.

Key Components

- **Argument Register (A):** Argument Register (A) — n-bit register; holds the search key (the content to search for)
- **Key Register (K):** Key Register (K) — n-bit comparator register; acts as a mask to specify which bits to compare
- **Memory Array:** Associative Memory Array ($m \times n$) — Storage area with m words, each of n bits; contains logic to compare
- **Match Register (M):** Match Register (M) — m-bit register; each bit corresponds to one memory word; set to 1 if match found

Working / Operation (Explanation)

29. CPU places required content in Argument Register 'A'
 30. Checking is done if and only if K-register values are '1'
 31. Required content is checked in Associative Memory using Key Register 'K'
 32. If required match is found → Match Register bit is set to '1'
 33. If not found → Match Register bit is set to '0'
- Note: A and K registers are n-bits wide; every operation is done in binary.

Numerical Example

Let A = 10111100, K = 11100000

Let Memory: Word1 = 10011110, Word2 = 10100001

- The first 3 bits of A are compared with Associative memory using K = '111'
- Word1: 100 ≠ 101 → No Match
- Word2: 101 = 101 → Match → M bit set to 1

Advantages

- Very high access speed — search done simultaneously for all words
- Used in cache memory tag comparison
- Used in virtual memory translation tables

Disadvantages

- Expensive — each cell needs comparison logic
- Complex hardware design

Q2a: Illustrate the Mapping Techniques in Cache Memory.

Cache Memory Background

When a cache miss occurs, data must be transferred from main memory to cache. The mapping technique determines how main memory blocks are placed in cache. Three techniques are used:

1. Associative Mapping

- Any block of main memory can be placed in ANY location in cache.
- The entire memory address is used as the TAG stored with data in cache.
- CPU Request Address is placed in Argument Register 'A' and compared with all TAG entries simultaneously.
- Cache HIT → data is taken; Cache MISS → data loaded from main memory.
- Disadvantage: Slow (checks all addresses line by line); makes the system slow.

2. Direct Mapping

- Address is divided into two fields: TAG (6 bits) and INDEX (9 bits).

- CPU first checks the INDEX, and if matched, checks the TAG.
- Both matched → corresponding data is taken.
- Faster than associative mapping (only 2-field check instead of full compare).
- Disadvantage: Multiple addresses with same INDEX but different TAGs cause conflicts → requires more time.

3. Set Associative Mapping

- Combines the best of both associative and direct mapping.
- Single index maps to multiple tags simultaneously (set of tags).
- Addresses with same index but multiple tags are executed in the same time period.
- Hit ratio is higher compared to misses → most efficient method.
- If cache is full, cache replacement algorithms are used: FIFO, LRU, Optimization.

Cache Replacement Algorithms

34. FIFO (First In First Out) — First inserted data is deleted first when cache is full
35. LRU (Least Recently Used) — Least recently accessed data is replaced
36. Optimization Algorithm — Replaces the data that will not be used for the longest time

Hit Ratio and Miss Ratio

- Hit Ratio = Number of Hits / (Number of Hits + Number of Misses)
- Miss Ratio = Number of Misses / (Hits + Misses) OR 1 - Hit Ratio
- Problem: 39 cache hits, 2 misses → Hit Ratio = $39/41 = 0.951$, Miss Ratio = $2/41 = 0.04$

Q2b: Describe Virtual Memory features and address translation.

Definition of Virtual Memory

Virtual Memory is an illusion memory that acts as a sub-part of main memory. It comes into existence when the main memory size is less than the program size that needs to be executed.

Key Features

- Allows programs larger than physical main memory to execute.
- OS manages transferring small parts of program between virtual and main memory.
- Auxiliary memory addresses are called Virtual Addresses.
- Main memory addresses are called Physical Addresses.
- Union of all virtual addresses = Address Space.
- Union of all physical addresses = Memory Space.
- Virtual Memory comes into existence only during memory limitation.

Configuration Example

- Main Memory (MM): $32 \text{ KB} = 2^5 \times 1\text{KB} = 2^5 \times 2^{10} = 2^{15} \rightarrow 15\text{-bit address}$
- Auxiliary Memory (AM): $1024 \text{ KB} = 2^{10} \times 1\text{KB} = 2^{20} \rightarrow 20\text{-bit address}$
- Auxiliary memory divided into pages; Main memory divided into blocks (same size)

Address Translation Process

37. Virtual Address (20-bit) is sent to Memory Mapping Table (MNT)
38. MNT accepts page number as input → produces block number as output
39. Memory Table Buffer Register (MTBR) stores block number → supplies to MMAR
40. Main Memory Address Register (MMAR) combines block number + line number → produces physical address

41. Physical address is used to access data in Main Memory

Page Fault Handling

- If main memory is full → condition called Page Fault occurs.
- To avoid page fault → use Replacement Algorithms: FIFO, LRU, Optimization.

Q3a: Explain how CPU is connected to main memory (RAM and ROM) with neat diagram.

RAM Block Diagram Description

- RAM chip connections: CS1, CS2 (chip select), RD (read), WR (write), AD7 (7-bit address), 8-bit data bus
- CS1 and CS2: used to select the RAM chip
- Configuration example: 128×8 RAM chip (128 memory words, 8 bits each)
- 8-bit data bus connects RAM to CPU/memory bus

ROM Block Diagram Description

- ROM chip connections: CS1, CS2 (chip select), RD (read), AD9 (9-bit address), 8-bit unidirectional data bus
- Configuration example: 512×8 ROM chip
- Only read operation is performed (no write line needed)
- 8-bit unidirectional data bus (data flows only from ROM to CPU)

Types of ROM

Type	Full Name	Key Feature
MROM	Masked ROM	Data written during manufacturing; cannot be changed
PROM	Programmable ROM	User can write data once after manufacturing
EPROM	Erasable Programmable ROM	Data erased using UV light and reprogrammed
EEPROM	Electrically Erasable Programmable ROM	Data erased and rewritten electrically without removing from system
Flash ROM	Flash Memory	Advanced EEPROM; data erased and written in blocks

CPU-Memory Interface

- CPU → Main Memory → Auxiliary Memory (hierarchy)
- Main memory acts as central storage unit, stores programs and data
- Made of integrated ICs; acts as interface between CPU and auxiliary memory

Q3b: Explain Handshaking Method in Asynchronous Data Transfer with neat diagram.

Asynchronous Data Transfer Overview

In asynchronous data transfer, data is transmitted between source and destination using different clock cycles (unlike synchronous which uses the same clock). Two methods: Strobe and Handshaking.

Handshaking Method

In handshaking, a data acceptance signal (acknowledgement) is given by the destination indicating that data has been received accurately.

A. Source Initiated Handshaking

42. Data bus is activated by source unit.
43. Valid data is present on the data bus.
44. Destination receives data and provides a Data Acceptance signal back to source.
45. After completion, D.B and V.D are set to low by source unit.
46. Destination unit disables Data Acceptance signal.

B. Destination Initiated Handshaking

47. Destination Unit (D.U) sends 'Ready to Receive' signal to Source Unit (S.U).
48. Data bus is activated by source.
49. Valid data is placed on data bus by S.U.
50. Destination disables 'Read for Data' signal immediately when source disables D.B and V.D.

Note: The flow of these operations is also called a Sequential Diagram.

Advantage over Strobe

- Strobe provides no acknowledgement to source whether destination received data or not.
- Handshaking overcomes this by providing a 'Data Acceptance' signal for confirmation.

Key Signals

- D.B = Data Bus
- V.D = Valid Data signal
- DA = Data Acceptance signal
- S = Source, D = Destination

Q4a: Illustrate the Daisy Chaining method of interrupt handling.

Interrupt Initiated I/O Background

To reduce CPU overload in Programmed I/O, Interrupt-Initiated I/O is used. When a device wants to transfer data, it raises an interrupt. CPU stops the main program, stores the next instruction in stack, and starts executing the interrupt service routine.

Daisy Chain Method (Hardware Approach)

- Devices are connected serially (one after another) in a chain.
- CPU decides the highest priority interrupt instead of using a software program.
- Devices are connected in descending order of priority.
- Device closest to CPU has highest priority; device farthest has lowest priority.

Key Signals

- INTR: Device → CPU (Interrupt Request — device asks CPU for service)
- INTA: CPU → Device (Interrupt Acknowledge — CPU confirms service)
- PI = Priority Input, PO = Priority Output
- VAD = Vector Address (unique address for each device)

Working Procedure

51. All devices send interrupt request to CPU via INTR line
52. CPU sends Interrupt Acknowledge (INTA) to the highest priority device (Device 1)

53. If Device 1 raised the interrupt: it activates VAD1 and blocks INTA to next device (PO = 0)
54. If Device 1 did NOT raise interrupt: it passes INTA to Device 2 via PO → PI chain
55. CPU executes the interrupt of the highest priority device using Vector Address
56. Devices are connected in descending order of priority (Device 0 = highest, Device n = lowest)

Disadvantage

- Time taken for finding the interrupt-initiated device is very high compared to time taken for executing the interrupt.
- To avoid this disadvantage → use Parallel Priority Interrupt method.

Q4b: Explain DMA (Direct Memory Access) mode of data transfer with neat diagram.

DMA Definition

In DMA, a DMA controller is used to transfer data between I/O devices and memory without CPU intervention. CPU intervention is '0' (zero), so CPU performance is very high compared to the above 2 methods (Programmed I/O and Interrupt I/O).

Components of DMA System

57. **CPU:** CPU — has address bus, data bus, Read/Write; signals: BR (Bus Request), BG (Bus Grant), INT (Interrupt Signal)
58. **DMA Controller:** DMA Controller — has address bus, data bus, Read/Write, BR, BG, INT; special signals: DS (DMA Select), RS (Register Select)
59. **Memory Block:** Memory Block (RAM) — has address bus, data bus, Read/Write; stores data to transfer to I/O device
60. **I/O Device:** I/O Device — contains: DMA Req (DMA Request) and DMA Ack (DMA Acknowledgement)

DMA Transfer Procedure

61. I/O device wants to send data to memory → sends DMA Request signal to DMA controller
 62. DMA controller sends Bus Request (BR) signal to CPU
 63. If CPU is in idle state → grants buses using Bus Grant (BG) signal
 64. Using granted buses, DMA controller transfers data from I/O to memory
 65. After transfer completion → INT signal sent by DMA to CPU indicating to take back granted buses
 66. DMA sends Acknowledgement signal to I/O devices indicating required operations are completed
- Note: If buses are NOT free → CPU sends Interrupt signal to DMA controller
 - All the buses are Bidirectional

DMA Methods

67. Cycle Stealing Method — Bit-by-bit transfer takes place from memory to I/O
68. Burst Method — Word of data transfer takes place from memory to I/O device

Conditions

- BG = 1 → Buses are granted, transfer takes place
- BG = 0 → Buses are not granted, no transfer takes place

Q5a: Explain Programmed I/O mode of data transfer with neat diagram.

Definition

In Programmed I/O (CPU-Initiated), the CPU controls all data transfer between I/O devices and memory. The CPU continuously checks the status of the I/O device before transferring data.

Components

- CPU — Contains Data Register (DR), Status Register, Flag Register (f); performs Read/Write operations
- Interface — Contains Data Register, Status Register with Flag bit (f)
- I/O Device — Provides Data Valid and Data Accepted signals

Working Procedure

69. CPU reads Data Register and checks Flag register.
70. If Flag (f) = 1 → Data transfer CAN be done.
71. If Flag (f) = 0 → NO data transfer is performed.
72. CPU reads Status Register and transfers data to memory.
73. Checks if operation is complete; if YES → continues with main program.

Flowchart Steps

- Read Data Register
- Check Flag bit → if 0, loop back and re-check
- If Flag = 1 → Read Status Register
- Transfer data to memory
- Operation complete? If NO → repeat; if YES → continue with program

Disadvantage

- Every time CPU has to check flag register value, besides its own working it consumes more time and creates heavy workload.
- Overall performance of the system decreases.
- To overcome → use Interrupt Initiated I/O.

Q5b: Describe DMA Controller and its operation with neat diagram.

DMA Controller Description

The DMA Controller is an interface between the I/O device and the system bus. It manages the data transfer without CPU intervention.

DMA Controller Signals

- Address Bus — To specify memory address for data transfer
- Data Bus — To carry the actual data
- Read (RD) — Read control signal
- Write (WR) — Write control signal
- BR (Bus Request) — DMA requests bus from CPU
- BG (Bus Grant) — CPU grants bus to DMA
- INT (Interrupt) — Sent after transfer completion to reclaim buses
- DS (DMA Select) — Selects specific DMA controller
- RS (Register Select) — Selects register within DMA controller

DMA Operation Steps

74. I/O device sends DMA Request to DMA controller

75. DMA controller asserts BR (Bus Request) to CPU
76. CPU grants buses via BG (Bus Grant) when idle
77. DMA takes control of system buses (address + data bus)
78. DMA transfers data directly from I/O to Memory (RAM)
79. After transfer: DMA sends INT to CPU to reclaim buses
80. DMA sends ACK to I/O device confirming operation completion

UNIT - V: Multi-Processors and Pipelining

Q1a: Discuss the features of Multiprocessor Systems.

Definition of Multiprocessors

A Multiprocessor is a computer system with two or more CPUs sharing common memory and I/O devices for parallel processing and high-speed execution.

Key Features / Characteristics

- **1. Reliability:** The failure of one CPU does not affect the overall system performance.
 - The work is distributed among different CPUs.
- **2. Scalability:** The number of CPUs can be increased based on user requirement.
 - System can expand to meet growing processing demands.
- **3. Improved Performance:** Multiple CPUs execute different types of instructions at the same time, so system performance increases.
 - Parallel execution reduces overall execution time.
- **4. Multiple Applications:** Can be used as SISD, SIMD, MISD, and MIMD — supports all parallel processing modes.

Types of Multiprocessors (Memory Organization)

81. Shared Memory Technique — Common memory is used for all CPUs

82. Distributed Memory Technique — Each CPU uses its own private memory

Parallel Processing Categories

Type	Full Name	Speed	Cost	Complexity	Description
SISD	Single Instruction Single Data	Lowest	Lowest	Lowest	Sequential; single CPU
SIMD	Single Instruction Multiple Data	Medium	Medium	Medium	One instruction, many data elements
MISD	Multiple Instruction Single Data	High	High	High	Multiple instructions, one data; pipeline
MIMD	Multiple Instruction Multiple Data	Highest	Highest	Highest	Multiple CPUs, multiple tasks

Speed: MIMD > MISD > SIMD > SISD | Cost: MIMD > MISD > SIMD > SISD

Q1b: Explain different Interconnection Structures used in Multiprocessor Systems.

Inter-Connection of Multiprocessors

Inter-connection provides the connection between CPU and memory modules. It is implemented by 3 methods:

1. Time-Shared Common Bus

- Common memory is shared among different CPUs using a single bus for communication.
- Single bus is only responsible for all communication.
- If CPU-1 wants to communicate with memory: remaining CPUs should be in waiting state until bus is free.
- Advantages: Simple to implement, low cost and complexity.

- Disadvantages: Data transfer is low; CPUs have to be in waiting state.

2. Multiport Memories

- Memory is divided into different modules along with CPUs.
- Different buses are used for connecting CPUs and every memory module.
- If CPU1 wants to communicate with MM1 AND CPU3 wants to communicate with MM3 — both can be done simultaneously at the same time.
- If CPU1 AND CPU3 both want to communicate with MM1 — CANNOT be done at the same time; priority system is used.
- High priority CPU communicates with memory first.
- Advantage: High speed data transfer.
- Disadvantage: Circuit complexity, high cost.

3. Crossbar Switch Network

- Multiple buses are replaced with crossbar switches.
- Switch is responsible for passing the request between CPU and memory units.
- Each CPU and memory module intersection has a crossbar switch.
- Switch allows simultaneous connections — any CPU can connect to any memory module.
- Advantage: High speed data transfer.
- Disadvantage: Circuit complexity, high cost.

Q2a: Illustrate the Time-Shared Common Bus and Multiport Memory Organization.

Time-Shared Common Bus

In this interconnection structure, a single bus connects all CPUs, I/O devices, and memory. Only one device can use the bus at a time.

- Common memory unit connects to: CPU-1, CPU-2, CPU-3, I/O P1, I/O P2 via single bus.
- If any CPU wants to communicate with memory, remaining CPUs must wait.
- Bus is a shared resource — access is time-multiplexed.

Multiport Memory Organization

Memory is divided into separate modules (MM1, MM2, MM3...) and each module has dedicated bus connections to all CPUs.

- CPU-1 can connect to MM1, MM2, or MM3 via separate buses.
- CPU-2 can independently connect to any MM while CPU-1 uses a different MM.
- Parallel communication is possible if different CPUs access different memory modules.
- Conflict occurs only when two CPUs try to access the same memory module simultaneously.

Comparison

Time-Shared Common Bus	Multiport Memory
Single bus for all CPUs	Dedicated buses for each memory module
One CPU communicates at a time	Multiple CPUs communicate simultaneously
Simple and low cost	Complex and high cost
Low data transfer rate	High data transfer rate

Q2b: Explain the Serial (Daisy Chain) Arbitration Technique with a neat diagram.

(Refer to Q4a of Unit IV for complete Daisy Chain explanation)

Daisy Chain Arbitration — Key Points

- Used in multiprocessor systems to resolve bus contention when multiple devices request the bus simultaneously.
- Devices are connected serially in a chain with priority determined by physical position.
- INTR: Device → CPU (Interrupt/Bus Request); INTA: CPU → Device (Acknowledgement)
- Each device has: PI (Priority Input) and PO (Priority Output) lines
- VAD (Vector Address) — unique address identifying each device

Serial Arbitration Procedure

83. All requesting devices raise the interrupt request line
84. CPU sends grant/acknowledge signal to Device 1 (highest priority)
85. If Device 1 requested bus → it uses the bus and blocks INTA from passing further (PO = 0)
86. If Device 1 did NOT request → it passes INTA to Device 2 via PO → PI connection
87. This continues until the requesting device receives acknowledgement

Priority Order

- Device 0 = Highest Priority (e.g., disk drive)
- Device 3 = Lowest Priority (e.g., keyboard)
- Devices connected in descending order of priorities

Q3a: Explain the Parallel Arbitration Technique using Priority Encoder and Decoder.

Definition

Parallel Priority Interrupt method is used to avoid the disadvantage of the daisy chain method. It uses hardware components (AND gates, priority encoder) to resolve multiple simultaneous interrupts.

Components

- I/O Devices — Each device connected to an interrupt register bit (I0, I1, I2, I3...)
- Interrupt Register — Holds interrupt status from each device
- Mask Register — Used to enable/disable specific interrupts; each device has a corresponding mask bit
- AND Gates — Combine interrupt register and mask register outputs
- Priority Encoder ($2^n \times n$) — Takes AND gate outputs and produces 2-bit priority code (x, y)
- VAD (Vector Address) — 8-bit output sent to CPU for interrupt identification
- IEN (Interrupt Enable) and IST (Interrupt Status) — control signals for AND gate outputs

Functional Table (Priority Encoder)

I0 I1 I2 I3	x	y	IST
1 x x x	0	0	1
0 1 x x	0	1	1
0 0 1 x	1	0	1
0 0 0 1	1	1	1
0 0 0 0	x	x	0

Working Procedure

88. Device raising interrupt → corresponding bit in interrupt register is set
89. Same location in mask register is also set high
90. Both are given to AND gates → output goes to priority encoder
91. Based on priorities, interrupts are resolved using CPU by taking the VAD
92. Whenever interrupt is generated: IEN → 1, IST → 1
93. IACK is used to know which device initiated the interrupt
 - Priority: Disk (highest) → Printer → Reader → Keyboard (lowest)

Advantage over Daisy Chain

- CPU workload is very less compared to programmed I/O.
- Performance of the system is better.
- All interrupts resolved simultaneously (parallel) — faster than serial daisy chain.

Q3b: Demonstrate the concept of Pipelining with a suitable example.

Definition of Pipelining

Pipelining is a technique used to build fast processors. It is a method where multiple instructions are executed at the same time by overlapping different stages of instruction execution.

Instruction Cycle Stages

94. FI (Fetch Instruction) — Processor fetches the next instruction to be executed
95. DI (Decode Instruction) — Processor decodes what type of instruction it is
96. EA/OF (Operand Fetch / Calculate Effective Address) — Find effective address / fetch operand values
97. EI (Execute Instruction) — Processor executes on data based on instruction

Pipeline Example with 3 Instructions

Consider: INST1, INST2, INST3 with stages F (Fetch), D (Decode), E (Execute)

Instruction	T0	T1	T2	T3
INST1	Fetch	Decode	OF/EA	Execute
INST2	—	Fetch	Decode	OF/EA
INST3	—	—	Fetch	Decode

- When INST1 is in Execution stage → INST2 is in Decoding stage → INST3 is in Fetch stage
- All three instructions execute simultaneously in different pipeline stages
- Each block is called a Pipe Stage or Pipe Segment

Disadvantages of Pipelining

98. Resource Conflicts — Occurs if initialization is not done properly
99. Data Dependence Conflicts — Occurs when data is initialized at last for operation
100. Branching Difficulty Conflicts — Occurs when iterative loops are used more in program

Q4a: Describe Instruction Pipeline with stages and explain with an example.

Instruction Pipeline

Instruction pipelining is used to build fast processors. Multiple instructions are executed simultaneously at different pipeline stages.

5 Stages of RISC Pipeline

101. FI (Fetch Instruction) — Processor fetches the next instruction from memory
102. DI (Decode Instruction) — Processor decodes instruction to determine operation type
103. OF (Operand Fetch) — Processor reads required values of data for operations
104. EI (Execute Instruction) — Processor executes on data based on instruction
105. RS (Result Storage) — Processor stores the result in register or memory

RISC Pipeline (5-stage): FI → DI → OF → EI → RS

Timing Diagram Example

Instr.	T0	T1	T2	T3	T4
INST-1	Fetch	Decode	OF/EA	Execute	RS
INST-2	—	Fetch	Decode	OF/EA	Execute
INST-3	—	—	Fetch	Decode	OF/EA

RISC Pipelining Features

- Simple instructions
- Executes in one clock cycle
- Fixed length instructions
- Few addressing modes
- High performance
- Also called Assembly-line Pipelining

Difference: Parallel Processing vs Pipelining

Parallel Processing	Pipelining
Multiple tasks executed simultaneously using more than one processor	A task is divided into stages; different stages work simultaneously
Many processors work together	One process is split into steps
Requires multiple processing units	Uses stages within a single processor
Ex: Multi-core computers, supercomputers	Ex: Instruction pipeline in CPU
Expensive hardware required	Delay may occur due to pipeline hazards

Q4b: Explain RISC Pipeline and discuss its operation with an example.

RISC — Reduced Instruction Set Computer

RISC pipelining is also called Assembly-line Pipelining. Every processor or controller follows RISC or CISC architecture, and the instruction set varies accordingly.

Characteristics of RISC

- Simple instructions
- Few addressing modes
- Instructions executed in one clock cycle
- Based on software (hardwired control)

- Fixed size instructions
- Transistors used for storing instructions
- Has 3-stage or 5-stage pipelining
- Higher speed than CISC

RISC Pipeline Stages (5-stage)

- 106. FI — Fetch Instruction
- 107. DI — Decode Instruction
- 108. OF — Operand Fetch
- 109. EI — Execute Instruction
- 110. RS — Result Storage

Pipeline Operation (3 Instructions Example)

- Consider instructions: INST1, INST2, INST3 with stages F, D, E
- INST1: F at T₀, D at T₁, E at T₂
- INST2: F at T₁, D at T₂, E at T₃
- INST3: F at T₂, D at T₃, E at T₄
- Time period = one clock cycle per stage

Examples of RISC Processors

- 8086, 8051, 8085 → follow RISC architecture
- ARM, ATMEL → follow CISC architecture (note: some sources differ)

Q5a: Explain the Attached Array Processor with a host computer using a block diagram.

Array Processor Definition

An Array Processor is responsible for storing data, performing required operations, and follows specific architecture for dealing with matrices and vectors.

Types of Array Processors

- 111. Attached Array Processor
- 112. SIMD Array Processor

Attached Array Processor — Working

- Works along with a main (host/general purpose) computer.
- Main CPU gives instructions to the array processor.
- Array processor performs heavy mathematical calculations.
- Results are sent back to the CPU.

Block Diagram Components

- General Purpose Computer ↔ I/O Interface ↔ Attached Array Processor
- Main Memory ← High Speed Memory-to-Memory Bus → Local Memory of Array Processor

Characteristics

- Connected to a host computer.
- Used for numerical calculations.
- Increases processing speed.
- Works as a supporting (co-)processor.
- Used for scientific calculations.

- Also used in weather forecasting.
- Array processor can be self-contained unit OR distributed unit connected to memory and I/O devices using buses.

Advantages

- Reduces load on main CPU.
- Faster mathematical computations.
- Efficient for scientific applications.

Disadvantages

- Expensive.
- Requires coordination with main CPU.

Q5b: Explain the SIMD Array Processor with a neat block diagram.

SIMD Definition

SIMD = Single Instruction Multiple Data. One instruction is applied to many data elements simultaneously. It is a type of array processor that follows parallel execution.

Key Characteristics

- One instruction is applied to many data elements simultaneously.
- A Master Control Unit sends the same instruction to all processing elements.
- Each processing element works on different data at the same time.
- Has parallel execution.
- Same instruction for multiple data.
- High speed computation.
- Multiple processing elements.

Block Diagram Components

- Master Control Unit — Sends same instruction to all Processing Elements (PE1, PE2, ... PEn)
- Processing Elements (PE1 to PEn) — Each processes different data simultaneously
- Memory Modules (M1 to Mn) — Each PE has its own local memory module
- Main Memory — Shared memory connected to Master Control Unit

SIMD vs Attached Array Processor

SIMD Array Processor	Attached Array Processor
Standalone array processor	Works with host computer
One instruction to all PEs simultaneously	Main CPU sends instructions, AP does calculations
Best for array/matrix operations	Best for scientific/numerical computation
Complex hardware	Expensive but simpler structure

Advantages of SIMD

- Very fast.
- Efficient for array operations.
- Better performance in image processing.

Disadvantages of SIMD

- Complex Hardware.
- Not suitable for sequential tasks.